

1. (16 points) Use the iteration method to solve the following recurrence equation.
 a) (8 points)

$$T(n) = \begin{cases} T(n-1) + n & \text{if } (n > 1) \\ 1 & \text{if } (n = 1) \end{cases}$$

Answer:

$$T(n) = T(n-1) + n = T(n-2) + n + n-1 = T(n-3) + n + n-1 + n-2 = \dots = T(1) + n + n-1 + n-2 + \dots + 2 = 1 + 2 + \dots + n-1 + n = n(n+1)/2 = \theta(n^2).$$

b) (8 points) $T(n) = \begin{cases} 0 & \text{if } n = 2 \\ T(\sqrt{n}) + 1 & \text{if } n > 2 \end{cases}$

Answer:

$$T(n) = T(\sqrt{n}) + 1 = T(n^{1/2}) + 1 = T(n^{1/2^2}) + 2 = T(n^{1/2^3}) + 3 = \dots = T(n^{1/2^k}) + k.$$

The iterative expansion process stops when $n^{1/2^k} = 2$ for some k . When this happens, $T(n) = T(2) + k = k$.

From $n^{1/2^k} = 2$ and take logarithm on both sides, we have $(1/2^k) \lg n = 1$, or $2^k = \lg n$.

Take logarithm on both sides of $2^k = \lg n$, we have $k = \lg \lg n$.

Therefore, $T(n) = \lg \lg n$.

2. (6 points) Use Master method to solve $T(n) = 4T(n/2) + n^2$ and $T(1)=1$.

Answer:

$$a=4, b=2, k=2. \text{ As } a=b^k, T(n)=\theta(n^k \lg n) = \theta(n^2 \lg n)$$

3. Professor Caesar wishes to develop a matrix-multiplication algorithm that is asymptotically faster than Strassen's Algorithm. His algorithm will use divide-and conquer method, dividing each matrix into pieces of size $n/4 \times n/4$, and the divide and combine steps together will take $\theta(n^2)$ time. He needs to determine how many subproblems his algorithm has to create in order to beat Strassen's algorithm. If his algorithm creates a subproblems, then the recurrence for the running time $T(n)$ becomes $T(n) = aT(n/4) + \theta(n^2)$. What is the largest integer value of a for which Professor Caesar's algorithm would be asymptotically faster than Strassen's algorithm? (10 points)

Answer:

Recall that the master theorem: $T(n) = aT(n/b) + cn^k$.

- $T(n) = \Theta(n^k)$ if $a < b^k$
- $T(n) = \Theta(n^k \lg n)$ if $a = b^k$
- $T(n) = \Theta(n^{\log_b a})$ if $a > b^k$

We know that for Strassen's algorithm, $T(n) = \theta(n^{\lg 7})$.

For the algorithm to be developed by Professor Caesar, $b=4, k=2$.

if $a > b^k$, $T(n) = \theta(n^{\log b^a}) = \theta(n^{\log 4^a})$.

To make Professor Caesar's algorithm asymptotically faster than Strassen's algorithm, we need:

$$\log_4 a < \lg 7 \quad \Rightarrow \quad a < 4^{\lg 7} = 2^{2\lg 7} = 2^{\lg 49} = 49.$$

Then the largest integer value for a is 48.

4. (15 points) We want to find the largest item in a list of n items.
- a) Use the divide-and-conquer approach to write an algorithm (pseudo code is OK). Your algorithm will return the largest item (you do not need to return the index for it). The function that you need to design is *int maximum (int low, int high)*, which *low* and *high* stands for low and high index in the array (8 points).
 - b) Analyze your algorithm and show its time complexity in order notation (using θ). (7 points).

Answer:

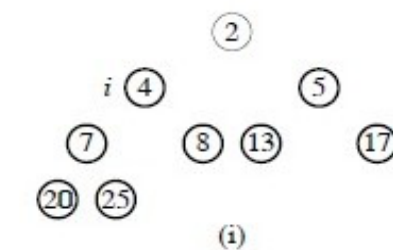
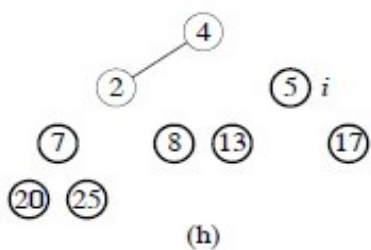
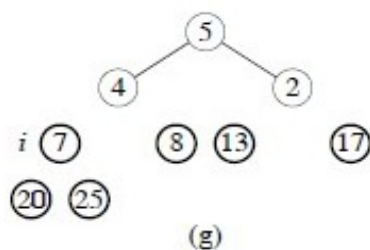
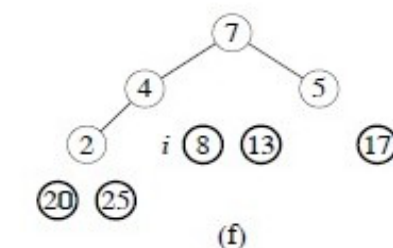
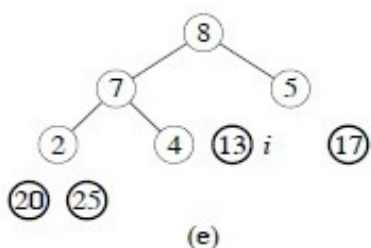
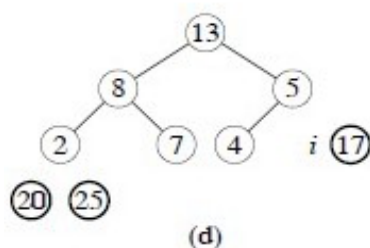
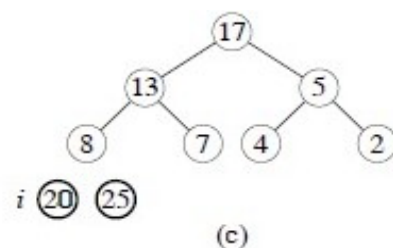
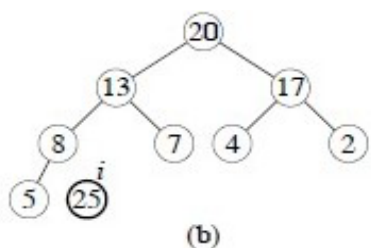
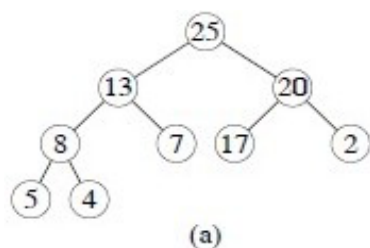
a)

```
int maximum (int low, int high) {  
    if (low == high)  
        return a[low];  
    int mid = (low + high) / 2; // integer division  
    int L1 = maximum(low, mid);  
    int L2 = maximum (mid+1, high);  
    if A[L1] > A[L2] then  
        return L1  
    else  
        return L2  
}
```

b) complexity analysis: $T(n) = 2T(n/2) + 1$, which yields $\theta(n)$.

5. (15 points) Illustrate the operation of Heapsort on the input array $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$. Draw the heap just after Build-Max-Heap was executed. Then draw a new heap after another (the next) element has been sorted; the last heap you draw has a single element.

Answer:



A

2	4	5	7	8	13	17	20	25
---	---	---	---	---	----	----	----	----

6. (15 points) Argue for the correctness of Heapsort using the following loop invariant: At the start of the iteration with an i of the for loop, (a) the subarray $A[1 \dots i]$ is a max-heap containing the i smallest elements of $A[1 \dots n]$, and (b) the subarray $A[i+1 \dots n]$ contains the $n - i$ largest elements of $A[1 \dots n]$ in correctly sorted order (i.e., in ascending order).

Answer:

We consider the three steps one by one.

- *Initialization:* Before the first iteration starts, $i = n$. At this time, $A[1 \dots i]$ contains all elements of A and $A[i+1 \dots n]$ is empty. Both (a) and (b) are trivially satisfied.
- *Maintenance:* Suppose the two conditions in the loop invariant hold at the start of the iteration with a given i . That is, that is, subarray $A[1 \dots i]$ is a max-heap containing the i smallest elements of $A[1 \dots n]$, and the subarray $A[i+1 \dots n]$ contains the $n - i$ largest elements of $A[1 \dots n]$ in correctly sorted order. During the iteration with this i , the algorithm first exchanges $A[1]$ with $A[i]$, where $A[1]$ was the largest element in $A[1 \dots i]$ before the exchange because it is a max-heap. After the exchange, $A[i]$ becomes the largest element among $A[1 \dots i]$, but it is still $\leq$ the elements in $A[i+1 \dots n]$. Now $A[i]$ will be added to the sorted subarray and removed from $A[1 \dots i]$ (by not being included in the next Max-Heapify), which will make $A[1 \dots i]$ one element fewer (i.e., the new i for the next iteration was actually $i - 1$ for the previous iteration) and $A[i+1 \dots n]$ one element more. Since $A[i]$ was the largest element among $A[1 \dots i]$ before its move, it is clear that both properties (a) and (b) still hold after this iteration.
- *Termination:* The “for loop” terminates when $i = 1$. In this case, $A[1]$ contains the smallest element of $A[1 \dots n]$ and the subarray $A[2 \dots n]$ contains the $n - 1$ largest elements of $A[1 \dots n]$ in correctly sorted order. This means that the entire array A is sorted correctly.